

Decisiones del Grupo

Proyecto 4: Muestra Inteligente — Una Encuesta Pequeña pero Equilibrada

Sebastián Gutiérrez, Mateo Bernal, Thomas Ramírez

Universidad Nacional de Colombia — Sede Manizales

Introducción a las Ciencias de la Computación

Profesor: Carlos Manuel Orrego Franco

17 de junio de 2026

Decisiones del Grupo

Pregunta 1

¿Qué representación eligieron y por qué?

Respuesta:

Representamos cada estudiante como un diccionario de Python con cuatro claves: *programa*, *semestre*, *ciudad*, *jornada*. La muestra parcial es una lista de esos diccionarios, y el pool completo también.

La alternativa que descartamos fue usar listas con índices posicionales — algo como `[0, 1, 2, 1]` para representar Computación, semestre 1, Manizales, Diurna. Eso es más compacto en memoria, pero obliga a mantener un mapeo constante entre índice y significado, lo cual encarece el debugging y hace que las funciones de restricción sean opacas.

La decisión tiene fundamento en Ciencias de la Computación: cuando la legibilidad de los datos determina la corrección del algoritmo, el costo de un poco de memoria adicional se amortiza completamente. En backtracking, donde las funciones `es_valido()` y `puede_podar()` inspeccionan atributos en cada llamada recursiva, tener `candidato["ciudad"]` en lugar de `candidato[2]` elimina una clase entera de errores silenciosos.

La consecuencia práctica: todas las funciones del algoritmo son autoexplicativas sin documentación adicional. El árbol de búsqueda del informe se construyó directamente con los nombres reales, no con índices abstractos.

Pregunta 2

¿Qué restricciones implementaron primero?

Respuesta:

Primero implementamos las restricciones máximas — específicamente `max_por_ciudad` — dentro de `es_valido()`. Después implementamos las restricciones mínimas — `min_programa`, `min_semestre_1`, `min_nocturna` — dentro de `puede_podar()`.

El orden no fue arbitrario: hay una asimetría fundamental entre cómo se verifican máximos y mínimos en backtracking. Un máximo se puede evaluar localmente en el momento de agregar un candidato: si ya hay 3 estudiantes de Manizales y el tope es 3, rechazamos el siguiente de Manizales de inmediato, sin mirar el futuro. Un mínimo, en cambio, requiere razonamiento prospectivo: para saber si voy a poder cumplir «al menos 2 de Computación», necesito contar cuántos de Computación quedan disponibles en el pool restante.

Esta distinción — restricciones que se verifican hacia atrás versus hacia adelante — es una propiedad estructural del backtracking, no un detalle de implementación. Confundirlos fue nuestro primer error conceptual: al inicio pusimos los mínimos en `es_valido()` y el algoritmo los ignoraba silenciosamente porque un solo candidato nunca viola un mínimo por sí solo.

Pregunta 3

¿Qué bug encontraron durante el desarrollo?

Respuesta:

Al probar el Conjunto 3 — `max_por_ciudad=2` con `tamano=7` — el algoritmo tardaba **8.380 llamadas recursivas** en concluir que no había solución. El comportamiento esperado era detectarlo en la primera llamada.

El problema era que nuestra versión inicial de `puede_podar()` verificaba si había suficientes candidatos válidos en general, pero no calculaba la capacidad total acumulada de todas las ciudades. Con 3 ciudades y tope de 2 por ciudad, la capacidad máxima es 6 plazas, pero pedíamos una muestra de 7. Esa contradicción es detectable antes de explorar una sola rama, pero la función no lo veía porque miraba ciudad por ciudad, no el sistema completo.

La solución fue añadir la **Condición 0**:

```
capacidad = suma de (max_ciudad - estudiantes_actuales_en_ciudad)
                para todas las ciudades disponibles
si capacidad < faltantes → podar
```

Eso convirtió 8.380 llamadas en **1 sola**. Es el ejemplo más claro del proyecto de cómo una poda bien diseñada no solo mejora la eficiencia — en este caso la diferencia entre explorar un árbol exponencial y detectar imposibilidad de raíz.

Pregunta 4

¿Qué caso de prueba falló inicialmente?

Respuesta:

El caso que falló fue precisamente el **Conjunto 3**: `tamano=7`, `max_por_ciudad=2`, tres ciudades disponibles (Manizales, Pereira, Armenia).

La falla no fue un resultado incorrecto — el algoritmo sí terminaba diciendo «sin solución». El fallo fue de **eficiencia**: sin la poda global, el árbol de búsqueda exploraba todas las formas posibles de seleccionar 7 estudiantes antes de rendirse, cuando la imposibilidad es demostrable matemáticamente en $O(1)$: $3 \text{ ciudades} \times 2 \text{ plazas} = 6 < 7$.

Esto es relevante en términos de complejidad computacional. El espacio de búsqueda de este problema es $C(20,7) = 77.520$ combinaciones en el peor caso. Sin la poda global, el algoritmo recorría una fracción significativa de ese espacio inútilmente. Con ella, la detección es inmediata sin importar el tamaño del pool.

Lo que el caso de prueba nos enseñó es que las invariantes globales del problema — aquellas que no dependen de ninguna elección individual — deben verificarse antes de iniciar cualquier exploración. En el dominio de satisfacción de restricciones esto se llama *arc consistency*; nosotros lo implementamos de forma específica para la restricción de ciudades.

Pregunta 5

¿Qué poda implementaron?

Respuesta:

Implementamos **poda por viabilidad futura** (*feasibility pruning*) en la función `puede_podar()`, con cinco condiciones independientes. El algoritmo poda una rama cuando detecta que es imposible completar la muestra desde el estado actual, aunque no se haya llegado al tamaño final.

Condición 0 — Capacidad global de ciudades: Si la suma de plazas disponibles en todas las ciudades es menor que los estudiantes que faltan, es imposible completar la muestra. Es la poda más fuerte y la que resolvió el bug del Conjunto 3.

Condición 1 — Candidatos insuficientes: Si los candidatos válidos restantes (ya filtrados por el tope de ciudad) son menos que los que faltan, nunca se completará la muestra.

Condiciones 2, 3 y 4 — Mínimos inalcanzables: Para cada restricción mínima (programa, semestre 1, jornada nocturna), si `actuales + disponibles_en_pool < mínimo_requerido`, la rama se descarta.

La separación entre `es_valido()` (poda local, verifica máximos al agregar) y `puede_podar()` (poda global, verifica viabilidad futura) es la decisión de diseño más importante del algoritmo. Sin `puede_podar()`, el backtracking se convierte en fuerza bruta con un filtro al final.

Pregunta 6

¿Qué cambiarían si tuvieran más tiempo?

Respuesta:

Tres cosas concretas.

Primero, usar un dataset real. Nuestro pool de 20 estudiantes es sintético y diseñado para que las soluciones sean alcanzables. Con datos reales de Bienestar Universitario aparecerían distribuciones desequilibradas — por ejemplo, muy pocos estudiantes nocturnos en ciertos programas — que harían las cuotas mínimas mucho más difíciles de satisfacer y pondrían a prueba la robustez de la poda.

Segundo, generalizar las restricciones. Actualmente `puede_podar()` tiene la Condición 0 escrita específicamente para la restricción de ciudad. Si el problema tuviera también un máximo por programa, habría que añadir otra condición análoga. Una versión más robusta representaría todas las restricciones como objetos con una interfaz común (`es_violada`, `capacidad_restante`), de forma que el motor de backtracking sea genérico y las restricciones sean enchufables.

Tercero, explorar *iterative deepening*. En lugar de fijar la profundidad máxima al tamaño de la muestra desde el inicio, *iterative deepening* aumenta la profundidad gradualmente. No mejoraría la primera solución (porque el espacio es pequeño), pero en versiones más grandes del problema permitiría encontrar soluciones más equilibradas con menor exploración promedio.

Pregunta 7

¿Qué parte hizo cada integrante?

Respuesta:

Sebastián Gutiérrez diseñó e implementó el algoritmo de backtracking completo: las siete funciones de `funciones.py`, el ciclo elegir-recursar-retroceder, y la lógica de las cinco

condiciones de poda. También fue quien diagnosticó y resolvió el bug del Conjunto 3 al identificar que la poda necesitaba una verificación de capacidad global, no solo local.

Mateo Bernal construyó el dataset sintético de 20 estudiantes, aseguró que la distribución del pool fuera realista (no trivialmente fácil ni imposible), generó las gráficas de comparación de distribuciones y de eficiencia del backtracking, e implementó la extensión de muestra óptima que encontró las 10.734 soluciones válidas del Conjunto 1 y calculó el score de balance.

Thomas Ramírez se encargó de los entregables de comunicación: la presentación en Beamer con colores UNAL y el árbol de búsqueda en TikZ, el informe escrito en LaTeX con las doce secciones requeridas, la demo interactiva en HTML/JavaScript que traduce el algoritmo al navegador sin dependencias externas, y la organización del repositorio.